



PennState

SWENG 894 Capstone

MicroMatch

Software Testing Report

Chastidy Joanem

M.S. Software Engineering | April 2026

[MicroMatch Repository Link](#)

Table of Contents

Table of Contents.....	1
1. Testing Approach and Tools.....	2
1.1 Testing Strategy Overview.....	3
1.2 Tools and Libraries.....	3
1.3 Repository and Test Files	3
2. Test Case Specification	4
2.1 Authentication and Account Management (FR-1, FR-2).....	5
2.2 Profile Management (FR-3, FR-4, FR-19).....	5
2.3 Project Management (FR-5, FR-6)	6
2.4 Application Workflow (FR-7, FR-8, FR-9).....	6
2.5 Deliverables and Project Completion (FR-10, FR-11, FR-12)	7
2.6 Feedback System (FR-13).....	8
2.7 Notifications, Messaging, Analytics (FR-16, FR-17, FR-18)	9
2.8 Smart Matching — Unit Tests (SA-1)	10
2.9 Administration, Logging, and E2E (FR-14, FR-15, E2E).....	10
3. Test Implementation	11
3.1 Automated Test Infrastructure	12
3.2 Unit Test Implementation — Matching Engine	12
3.3 Integration (API) Test Implementation.....	12
3.4 E2E System Test Implementation	12
4. Test Results	13
4.1 Sprint-by-Sprint Test Progress	14
4.1.1 Coverage Progress by Sprint.....	14
4.2 Code Coverage Analysis	14
4.3 Summary Test Results.....	15
5. Requirements Traceability Matrix.....	15
6. Defects and Observations	19

1. Testing Approach and Tools

MicroMatch employs a three-tier automated testing strategy — unit tests, integration (API) tests, and end-to-end (E2E) system tests — all executed against a SQLite test database to ensure no dependency on a running PostgreSQL instance. Tests are written with pytest and coverage measured by pytest-cov.

1.1 Testing Strategy Overview

Test Type	Scope	Tools / Framework	Database	Sprints
Unit Tests	Individual functions, scoring factors, badge logic, pure-function engine	pytest, pytest-cov, SQLite in-memory	SQLite (no ORM)	1–4
Integration (API) Tests	FastAPI endpoints via TestClient — request/response contracts, RBAC, DB persistence	pytest, FastAPI TestClient, SQLAlchemy SQLite	SQLite file (test.db)	1–4
E2E System Tests	Full lifecycle workflows: register, post, apply, accept, deliver, complete, feedback, admin logs	pytest, FastAPI TestClient (multi-step)	SQLite file (test.db)	1–4

1.2 Tools and Libraries

- pytest 7.x — test runner, fixtures, parameterization
- pytest-cov — coverage measurement and reporting
- FastAPI TestClient (httpx) — HTTP-level API testing without a running server
- SQLAlchemy + SQLite — test-scoped database mirroring production schema
- unittest.mock (patch) — isolates logging middleware from test DB
- Python-JOSE + bcrypt — JWT generation helpers in test auth fixtures

1.3 Repository and Test Files

All test source files: [backend/tests/](#)

- test_auth.py, test_auth_token_utils.py — FR-1, FR-2
- test_student_profile.py, test_organization_profile.py — FR-3, FR-4, FR-19
- test_project_api.py, test_project_retrieval.py — FR-5, FR-6
- test_apply_endpoint.py, test_application_status_update.py, test_get_my_applications.py — FR-7, FR-8, FR-9
- test_deliverables.py, test_deliverable_review.py, test_mark_project_complete.py — FR-10, FR-11, FR-12
- test_feedback.py — FR-13
- test_notifications.py, test_notification_api.py — FR-16
- test_messages.py — FR-17
- test_analytics.py — FR-18

- `test_profile_enhancements.py`, `test_badges.py` — FR-19
- `test_matching_engine.py`, `test_recommendations.py` — SA-1
- `test_logging_middleware.py`, `test_admin_moderation.py` — FR-15, FR-14
- `test_e2e_workflow.py`, `test_e2e_completion_workflow.py` — Full lifecycle E2E

2. Test Case Specification

Formal test case specifications for each functional requirement. Each entry includes test ID, requirement, description, preconditions, postconditions, expected result, and acceptance criteria.

2.1 Authentication and Account Management (FR-1, FR-2)

Test ID	Req.	Description	Preconditions	Postconditions	Expected Result	Acceptance Criteria
TC-US1-01	FR-1	Register new account with valid credentials	No existing account with that email	User record in DB; JWT returned	HTTP 201; token in response	Duplicate email returns 400
TC-US1-02	FR-1	Duplicate email registration blocked	Account exists with email	No new record created	HTTP 400	Email uniqueness enforced
TC-US1-03	FR-1	Login with valid credentials	Account exists	JWT token issued	HTTP 200; access_token present	Token valid for protected endpoints
TC-US1-04	FR-2	Student cannot access org-only endpoint	Student JWT	No data change	HTTP 403 Forbidden	RBAC enforced at endpoint
TC-US1-05	FR-2	Unauthenticated request blocked	No token	No data accessed	HTTP 401	401 for all protected routes
TC-US1-06	FR-2	Admin cannot post projects	Admin JWT	No project created	HTTP 403	Role isolation correct

2.2 Profile Management (FR-3, FR-4, FR-19)

Test ID	Req.	Description	Preconditions	Postconditions	Expected Result	Acceptance Criteria
TC-US2-01	FR-3	Student creates profile with all fields	Student logged in; no profile exists	Profile persisted in DB	HTTP 201; all fields returned	Readable via GET /student/profile
TC-US2-02	FR-3	Student updates existing profile	Profile exists	Fields updated	HTTP 200; updated values	Changes persisted to DB
TC-US3-01	FR-4	Org creates organization profile	Org logged in; no profile	Profile saved	HTTP 201	Visible on project listings
TC-US19-01	FR-19	Badges awarded after first project completion	Student has 1 completed project	Badges field updated	first_project badge present	Cannot be manually set
TC-US19-02	FR-19	PUT enhance ignores badges field	Student logged in	No badge change	badges field ignored	System-only badge assignment

TC-US19-03	FR-19	top Rated badge at avg rating >= 4.5	Student avg rating >= 4.5	top Rated badge set	top Rated in badges string	Live DB computation
------------	-------	--------------------------------------	---------------------------	---------------------	----------------------------	---------------------

2.3 Project Management (FR-5, FR-6)

Test ID	Req.	Description	Preconditions	Postconditions	Expected Result	Acceptance Criteria
TC-US4-01	FR-5	Org posts a new project listing	Org logged in; profile exists	Project in DB with status=open	HTTP 201; project_id returned	Visible in GET /projects
TC-US4-02	FR-5	Student cannot post projects	Student JWT	No project created	HTTP 403	RBAC enforced
TC-US5-01	FR-6	Student retrieves all open projects	At least one open project	Projects returned	HTTP 200; non-empty list	All open projects included
TC-US5-02	FR-6	Keyword search filters results	Multiple projects, different skills	Filtered results	HTTP 200; matching only	AND logic across keywords
TC-US5-03	FR-6	Search returns empty list for no match	No matching project	Empty list	HTTP 200; []	No 404 on empty

2.4 Application Workflow (FR-7, FR-8, FR-9)

Test ID	Req.	Description	Preconditions	Postconditions	Expected Result	Acceptance Criteria
TC-US6-01	FR-7	Student applies to open project	Student + open project exist	Application created, status=pending	HTTP 201	UNIQUE on student+project
TC-US6-02	FR-7	Duplicate application blocked	Student already applied	No new application	HTTP 400	IntegrityError caught
TC-US8-01	FR-8	Student retrieves their applications	Student has 1+ applications	List returned	HTTP 200; status field	Only that student's applications
TC-US9-01	FR-9	Org accepts an application	Pending application; org owns project	Status=accepted; project=closed	HTTP 200; status=accepted	Notification sent to student
TC-US9-02	FR-9	Org rejects an application	Pending application exists	Status=rejected	HTTP 200; status=rejected	Student notified
TC-US9-03	FR-9	Non-owning org cannot update status	Application for another org's project	No change	HTTP 403	Ownership enforced
TC-US9-04	FR-9	Cannot re-update non-pending application	Application already accepted	No change	HTTP 400	Transition guard: pending-only
TC-US9-05	FR-9	Other pending apps auto-rejected on accept	Project has 3 pending applicants	1 accepted; 2 rejected	HTTP 200	Auto-reject in DB confirmed

2.5 Deliverables and Project Completion (FR-10, FR-11, FR-12)

Test ID	Req.	Description	Preconditions	Postconditions	Expected Result	Acceptance Criteria
TC-US10-01	FR-10	Accepted student submits deliverable	Application status=accepted	Deliverable in DB, status=submitted	HTTP 201	One per application
TC-US10-02	FR-10	Duplicate deliverable blocked	Deliverable already exists	No second deliverable	HTTP 400	UNIQUE on application_id
TC-US10-03	FR-10	Pending applicant cannot submit	Application status=pending	No deliverable	HTTP 400	Accepted-only guard
TC-US10-04	FR-10	Empty deliverable content rejected	Application accepted	No deliverable	HTTP 400	Content validation
TC-US11-01	FR-11	Org accepts a deliverable	Deliverable status=submitted	status=accepted	HTTP 200; status=accepted	Student notified

TC-US11-02	FR-11	Org rejects a deliverable	Deliverable submitted	status=rejected	HTTP 200; status=rejected	Student notified
TC-US11-03	FR-11	Cannot review deliverable twice	Already reviewed	No change	HTTP 400	Idempotency guard
TC-US12-01	FR-12	Org marks project complete	Accepted deliverable exists	Project=completed; badges awarded	HTTP 200; status=completed	award_badges() called; student notified
TC-US12-02	FR-12	Cannot complete without accepted deliverable	No accepted deliverable	No change	HTTP 400	Completion guard enforced

2.6 Feedback System (FR-13)

Test ID	Req.	Description	Preconditions	Postconditions	Expected Result	Acceptance Criteria
TC-US13-01	FR-13	Submit feedback for completed project	Project status=completed	Feedback row in DB	HTTP 201; rating stored	UNIQUE(user_id, project_id)
TC-US13-02	FR-13	Feedback blocked for non-completed project	Project status=open	No feedback	HTTP 400	Completion guard
TC-US13-03	FR-13	Duplicate feedback prevented	Feedback already submitted	No duplicate	HTTP 400	One per user per project
TC-US13-04	FR-13	Rating out of range rejected	Project completed	No feedback	HTTP 422	Pydantic: 1-5 range
TC-US13-05	FR-13	Two users submit feedback for same project	Project completed; two users	Two feedback rows	HTTP 201 both	Different user_ids allowed

2.7 Notifications, Messaging, Analytics (FR-16, FR-17, FR-18)

Test ID	Req.	Description	Preconditions	Postconditions	Expected Result	Acceptance Criteria
TC-US16-01	FR-16	Notification created on application accepted	Status changed to accepted	Notification row for student	Notification in DB	Same DB transaction
TC-US16-03	FR-16	GET /notifications returns user's list	Notifications exist	List returned	HTTP 200	Only current user's
TC-US16-04	FR-16	Mark notification as read	Unread notification exists	is_read=true	HTTP 200	Idempotent
TC-US17-01	FR-17	Accepted student sends message	Application accepted	Message persisted	HTTP 201	Content and sender_id stored
TC-US17-03	FR-17	Non-participant blocked from messaging	User not accepted/not org	No message	HTTP 403	require_project_participant
TC-US17-04	FR-17	Messages returned oldest-first	Messages exist	Ordered list	HTTP 200; ascending	created_at ASC
TC-US18-01	FR-18	Student analytics	Student logged in	Metrics object	HTTP 200; 4 fields	SQL aggregate per request

		returns 4 metrics				
TC-US18-02	FR-18	Org analytics returns 4 metrics	Org logged in	Metrics object	HTTP 200; 4 fields	Cross-role returns 403

2.8 Smart Matching — Unit Tests (SA-1)

Test ID	Req.	Description	Preconditions	Postconditions	Expected Result	Acceptance Criteria
TC-SA1-01	SA-1	Perfect skill overlap returns 100.0	Matching skills	skill_score=100	100.0	Pure function; no DB
TC-SA1-02	SA-1	No skill overlap returns 0.0	Disjoint skills	skill_score=0	0.0	Intersection-over-required
TC-SA1-03	SA-1	No required skills returns neutral 50	Empty required_skills	skill_score=50	50.0	Neutral for open projects
TC-SA1-04	SA-1	New student success score is neutral	Zero history	success_score=50	50.0	New students not penalised
TC-SA1-06	SA-1	rank_projects returns descending order	Multiple ProjectDTOs	Sorted list	scores[0]>=scores[1]	O(N log N) sort
TC-SA1-07	SA-1	Determinism: same inputs = same output	Identical DTOs	Two identical lists	r1 == r2	Pure function
TC-SA1-08	SA-1	Endpoint returns 403 for non-students	Org JWT	No data	HTTP 403	require_role('student')
TC-SA1-09	SA-1	Endpoint returns 404 if no profile	Student; no profile	Error	HTTP 404	Profile required

2.9 Administration, Logging, and E2E (FR-14, FR-15, E2E)

Test ID	Req.	Description	Preconditions	Postconditions	Expected Result	Acceptance Criteria
TC-US15-01	FR-15	Every request produces a SystemLog row	Any request made	SystemLog in DB	Log: endpoint, method, status	user_id=null for unauthenticated
TC-US14-01	FR-14	Admin retrieves	Admin JWT	Logs list returned	HTTP 200	Non-admin returns 403

		system logs				
TC-E2E-01	FR-1–FR-9	Sprint 1-2 full workflow: register, post, apply, accept	Clean DB	All transitions verified	All assertions pass	DB verified at each step
TC-E2E-02	FR-1–FR-19	Sprint 3-4 full lifecycle: deliver, review, complete, feedback, admin logs	Clean DB	Entire lifecycle completed	All HTTP assertions pass	14-step; DB verified at each

3. Test Implementation

3.1 Automated Test Infrastructure

All automated tests use pytest with a shared conftest.py providing two key fixtures:

- client — a FastAPI TestClient backed by a SQLite in-memory session for API-level testing
- db_session — a direct SQLAlchemy session to the same SQLite DB for E2E state verification

The test database is created fresh per session via `Base.metadata.create_all(engine)`, ensuring complete isolation without requiring PostgreSQL.

3.2 Unit Test Implementation — Matching Engine

File: [backend/tests/test_matching_engine.py](#) | 54 unit tests | 100% branch coverage

The matching engine is a pure Python module with no SQLAlchemy dependencies, making all factor functions directly importable and testable:

```
from app.services.matching_engine import (
    parse_skills, compute_skill_match, compute_experience_match,
    compute_activity_score, compute_interest_match, compute_success_score,
    score_student_project, rank_projects, StudentDTO, ProjectDTO
)
```

Fixtures define a `base_student` (experienced, skilled) and `new_student` (zero history). Every factor is tested independently, then a composite test verifies the weighted formula directly.

3.3 Integration (API) Test Implementation

API tests use helper functions to create test users, projects, and applications directly in the DB, then invoke endpoints via TestClient. Each test uses unique emails to prevent cross-test contamination:

```
def test_submit_deliverable_success(client):
    db = TestingSessionLocal()
    student = create_test_user(db, 'student@test.com', 'student')
    application = create_test_application(db, student, project,
    status='accepted')
    response = client.post(f'/applications/{app.id}/deliverables',
        json={'content': 'Final link'}, headers=get_auth_headers(student))
    assert response.status_code == 201
```

3.4 E2E System Test Implementation

File: [backend/tests/test_e2e_completion_workflow.py](#) | TC-E2E-02

The E2E test registers all three roles then executes a 14-step sequence. DB state is verified after each critical transition via `db_session.expire_all()`:

- After accept: `Application.status == 'accepted'` AND `Project.status == 'closed'`
- After deliverable submitted: `Deliverable.status == 'submitted'`
- After deliverable accepted: `Deliverable.status == 'accepted'`
- After project completed: `Project.status == 'completed'`
- After both feedbacks: `len(feedback_entries) == 2`
- Admin log check: `/projects`, `/applications`, `/feedback`, `/deliverables` all logged; status 201 and 400 captured

The logging middleware is patched via `unittest.mock.patch` to write to the test SQLite DB, ensuring log writes are queryable in the same session.

4. Test Results

4.1 Sprint-by-Sprint Test Progress

Sprint	Tests Added	Cumulative	Modules Covered
Sprint 1	54	54	Auth, RBAC, Student Profile, Org Profile, Projects
Sprint 2	71	125	Applications: apply, track, accept/reject; full integration edge cases
Sprint 3	49	174	Deliverables, Deliverable Review, Project Completion, Feedback, Admin, Logging, E2E
Sprint 4	77	251	Notifications, Messaging, Analytics, Profile Enhancements, Badges, Matching Engine, Recommendations, E2E-02
Total	251	251	35 test files; all FR and SA-1 covered

4.1.1 Coverage Progress by Sprint

Sprint	Tests Added	Cumulative Tests	Coverage Achieved
Sprint 1	54	54	~96% on Sprint 1 modules (auth, profiles, projects)
Sprint 2	71	125	~96% cumulative (applications, RBAC edge cases added)
Sprint 3	49	174	~96% cumulative (deliverables, feedback, logging, E2E)
Sprint 4	77	251	≥ 96% overall — matching engine 100%, badges 100%
Total	251	251	≥ 96% overall across all 35 test files

4.2 Code Coverage Analysis

Coverage measured via pytest-cov against all non-test application modules:

Module	Statements	Coverage	Notes
app/core/auth.py	20	100%	Full coverage
app/core/dependencies.py	25	92%	Lines 34, 43 — edge-case token decode paths
app/models.py	58	100%	Full coverage
app/routers/auth.py	35	100%	Full coverage
app/routers/applications.py	53	98%	Line 192 — low-probability constraint path
app/routers/projects.py	43	95%	Lines 78–85 — pagination boundary
app/services/matching_engine.py	~110	100%	All 5 scoring factors verified
app/utils/badges.py	~30	100%	All 5 badge definitions verified
OVERALL	1350+	≥ 96%	13 lines missed across 3 low-risk branches

4.3 Summary Test Results

All 251 automated tests pass. No failures or bugs were observed in the final test run. The E2E tests confirm all inter-module integrations — RBAC, notifications, badge computation, deliverable guards, and logging — function correctly as a complete system.

Minor coverage gaps (92–95%) exist only on error branches: malformed JWT decode paths in `dependencies.py` and a pagination boundary in `projects.py`. These have no business logic and are low-risk.

5. Requirements Traceability Matrix

Maps every test ID to requirement ID, test type, sprint, pass/fail result, execution method, test file, and description. All tests are automated.

Test ID	Req. ID	Type	Sprint	Result	Execution	Test File	Description	Date Added	Last Updated
TC-US 1-01	FR-1	Integration	Sprint 1	PASS	Automated	test_auth.py	Register → HTTP 201, JWT returned	Jan 28, 2026	Jan 28, 2026
TC-US 1-02	FR-1	Integration	Sprint 1	PASS	Automated	test_auth.py	Duplicate email → HTTP 400	Jan 28, 2026	Jan 28, 2026
TC-US 1-03	FR-1	Integration	Sprint 1	PASS	Automated	test_auth.py	Login → HTTP 200, access_token present	Jan 28, 2026	Jan 28, 2026
TC-US 1-04	FR-2	Integration	Sprint 1	PASS	Automated	test_dependencies.py	Student on org endpoint → HTTP 403	Jan 28, 2026	Jan 28, 2026
TC-US 1-05	FR-2	Integration	Sprint 1	PASS	Automated	test_dependencies.py	No token → HTTP 401	Jan 28, 2026	Jan 28, 2026
TC-US 2-01	FR-3	Integration	Sprint 1	PASS	Automated	test_student_profile.py	Create student profile → HTTP 201, all fields saved	Jan 28, 2026	Jan 28, 2026
TC-US 2-02	FR-3	Integration	Sprint 1	PASS	Automated	test_student_profile.py	Update student profile → HTTP 200, persisted	Jan 28, 2026	Jan 28, 2026
TC-US 3-01	FR-4	Integration	Sprint 1	PASS	Automated	test_organization_profile.py	Create org profile → HTTP 201	Jan 28, 2026	Jan 28, 2026
TC-US 4-01	FR-5	Integration	Sprint 1	PASS	Automated	test_project_api.py	Org posts project → HTTP 201, status=open	Jan 28, 2026	Jan 28, 2026
TC-US 4-02	FR-5	Integration	Sprint 1	PASS	Automated	test_project_api.py	Student posts project → HTTP 403	Jan 28, 2026	Jan 28, 2026
TC-US	FR-6	Integration	Sprint 1	PASS	Automated	test_project_retrieval.py	GET /projects → open projects; keyword search	Jan 28,	Jan 28,

5-01							works	2026	2026
TC-US 5-03	FR-6	Integration	Sprint 1	PASS	Automated	test_project_retrieval.py	No match keyword → empty list, HTTP 200	Jan 28, 2026	Jan 28, 2026
TC-US 6-01	FR-7	Integration	Sprint 2	PASS	Automated	test_apply_endpoint.py	Student applies → HTTP 201, status=pending	Feb 22, 2026	Feb 22, 2026
TC-US 6-02	FR-7	Integration	Sprint 2	PASS	Automated	test_apply_endpoint.py	Duplicate application → HTTP 400	Feb 22, 2026	Feb 22, 2026
TC-US 8-01	FR-8	Integration	Sprint 2	PASS	Automated	test_get_my_applications.py	GET /applications/me → applications with status	Feb 22, 2026	Feb 22, 2026
TC-US 9-01	FR-9	Integration	Sprint 2	PASS	Automated	test_application_status_update.py	Accept → accepted, project=closed, notification	Feb 22, 2026	Feb 22, 2026
TC-US 9-02	FR-9	Integration	Sprint 2	PASS	Automated	test_application_status_update.py	Reject → rejected, student notified	Feb 22, 2026	Feb 22, 2026
TC-US 9-03	FR-9	Integration	Sprint 2	PASS	Automated	test_organization_view_applications.py	Non-owning org PATCH → HTTP 403	Feb 22, 2026	Feb 22, 2026
TC-US 9-05	FR-9	Integration	Sprint 2	PASS	Automated	test_application_status_update.py	Accept 1 → others auto-rejected in DB	Feb 22, 2026	Feb 22, 2026
TC-US 10-01	FR-10	Integration	Sprint 3	PASS	Automated	test_deliverables.py	Submit deliverable → HTTP 201, status=submitted	Mar 31, 2026	Mar 31, 2026
TC-US 10-02	FR-10	Integration	Sprint 3	PASS	Automated	test_deliverables.py	Duplicate deliverable → HTTP 400	Mar 31, 2026	Mar 31, 2026
TC-US 10-03	FR-10	Integration	Sprint 3	PASS	Automated	test_deliverables.py	Pending applicant submits → HTTP 400	Mar 31, 2026	Mar 31, 2026
TC-US 10-04	FR-10	Integration	Sprint 3	PASS	Automated	test_deliverables.py	Empty content → HTTP 400	Mar 31, 2026	Mar 31, 2026
TC-US 11-01	FR-11	Integration	Sprint 3	PASS	Automated	test_deliverable_review.py	Org accepts deliverable → accepted, student notified	Mar 31, 2026	Mar 31, 2026

TC-US 11-03	FR - 11	Integration	Sprint 3	PASS	Automated	test_deliverable_review.py	Cannot review twice → HTTP 400	Mar 31, 2026	Mar 31, 2026
TC-US 12-01	FR - 12	Integration	Sprint 3	PASS	Automated	test_mark_project_complete.py	Mark complete → completed, badges awarded, notified	Mar 31, 2026	Mar 31, 2026
TC-US 12-02	FR - 12	Integration	Sprint 3	PASS	Automated	test_mark_project_complete.py	No accepted deliverable → HTTP 400	Mar 31, 2026	Mar 31, 2026
TC-US 13-01	FR - 13	Integration	Sprint 3	PASS	Automated	test_feedback.py	Submit feedback for completed project → HTTP 201	Mar 31, 2026	Mar 31, 2026
TC-US 13-02	FR - 13	Integration	Sprint 3	PASS	Automated	test_feedback.py	Feedback for open project → HTTP 400	Mar 31, 2026	Mar 31, 2026
TC-US 13-03	FR - 13	Integration	Sprint 3	PASS	Automated	test_feedback.py	Duplicate feedback → HTTP 400	Mar 31, 2026	Mar 31, 2026
TC-US 15-01	FR - 15	Integration	Sprint 3	PASS	Automated	test_logging_middleware.py	Every request → SystemLog row with correct fields	Mar 31, 2026	Mar 31, 2026
TC-US 14-01	FR - 14	Integration	Sprint 3	PASS	Automated	test_admin_moderation.py	Admin GET /admin/logs → logs; non-admin → 403	Mar 31, 2026	Mar 31, 2026
TC-US 16-01	FR - 16	Integration	Sprint 4	PASS	Automated	test_notification_api.py	Notification on accept; GET /notifications list	Apr 17, 2026	Apr 24, 2026
TC-US 16-04	FR - 16	Integration	Sprint 4	PASS	Automated	test_notification_api.py	Mark as read → is_read=true, idempotent	Apr 17, 2026	Apr 24, 2026
TC-US 17-01	FR - 17	Integration	Sprint 4	PASS	Automated	test_messages.py	Accepted student sends message → HTTP 201	Apr 17, 2026	Apr 24, 2026
TC-US 17-03	FR - 17	Integration	Sprint 4	PASS	Automated	test_messages.py	Non-participant → HTTP 403	Apr 17, 2026	Apr 24, 2026
TC-US 18-01	FR - 18	Integration	Sprint 4	PASS	Automated	test_analytics.py	Student analytics → 4 correct metrics	Apr 17, 2026	Apr 24, 2026
TC-US	FR -	Integration	Sprint 4	PASS	Automated	test_analytics.py	Org analytics → 4 correct metrics	Apr 17,	Apr 24,

18-02	18							2026	2026
TC-US-19-01	FR-19	Integration	Sprint 4	PASS	Automated	test_badges.py	Badges awarded on completion; system-only	Apr 17, 2026	Apr 24, 2026
TC-SA-1-01	SA-1	Unit	Sprint 4	PASS	Automated	test_matching_engine.py	Perfect skill overlap → skill_score = 100.0	Apr 17, 2026	Apr 24, 2026
TC-SA-1-03	SA-1	Unit	Sprint 4	PASS	Automated	test_matching_engine.py	No required skills → skill_score = 50.0 (neutral)	Apr 17, 2026	Apr 24, 2026
TC-SA-1-04	SA-1	Unit	Sprint 4	PASS	Automated	test_matching_engine.py	New student → success_score = 50.0 (neutral)	Apr 17, 2026	Apr 24, 2026
TC-SA-1-06	SA-1	Unit	Sprint 4	PASS	Automated	test_matching_engine.py	rank_projects → descending match_score order	Apr 17, 2026	Apr 24, 2026
TC-SA-1-07	SA-1	Unit	Sprint 4	PASS	Automated	test_matching_engine.py	Determinism: identical inputs → identical output	Apr 17, 2026	Apr 24, 2026
TC-SA-1-08	SA-1	Integration	Sprint 4	PASS	Automated	test_recommendations.py	Non-student on /recommendations → HTTP 403	Apr 17, 2026	Apr 24, 2026
TC-SA-1-09	SA-1	Integration	Sprint 4	PASS	Automated	test_recommendations.py	Student without profile → HTTP 404	Apr 17, 2026	Apr 24, 2026
TC-E2E-01	FR-1-FR-9	E2E	Sprint 2	PASS	Automated	test_e2e_workflow.py	Sprint 1-2 full lifecycle; DB verified at each step	Feb 22, 2026	Feb 22, 2026
TC-E2E-02	FR-1-FR-19	E2E	Sprint 4	PASS	Automated	test_e2e_completion_workflow.py	14-step lifecycle: deliver→review→complete→feedback→admin	Apr 17, 2026	Apr 24, 2026

6. Defects and Observations

No defects, failures, or bugs were observed in the final test run. All 251 automated tests pass. The table below describes minor known coverage gaps that do not affect production correctness:

Location	Type	Lines Missed	Notes
app/core/dependencies.py	Edge-case branch	34, 43	Malformed JWT decode paths; tested manually
app/routers/projects.py	Pagination boundary	78–85	skip/limit edge case; primary flows fully covered
app/routers/applications.py	Error branch	192	Low-probability constraint path; core logic 100% covered